

Detecting Palindromes

A *palindrome* is a word or phrase that is spelled the same when written forward and backward. *Level*, *race car*, *taco cat*, and *a man, a plan, a canal . . . Panama* are all examples of palindromes. If you would like to detect whether a string is a palindrome, you can write a recursive `isPalindrome()` function.

The base case is a zero- or one-character string, which by its nature is always the same, whether forward or backward. We'll use an approach similar to the head-tail technique, except that we'll split the string argument into head, middle, and last strings instead. If the head and last characters are the same and the middle characters also form a palindrome, the string is a palindrome. The recursion comes from passing the middle string to `isPalindrome()`.

Let's ask the three recursive algorithm questions about the `isPalindrome()` function:

What is the base case? A zero- or one-character string, which returns `True` because it is always a palindrome.

What argument is passed to the recursive function call? The middle characters of the string argument.

How does this argument become closer to the base case? The string argument shrinks by two characters for each recursive call until it becomes a zero- or one-character string.

Here is *palindrome.py*, a Python program to detect palindromes:

Python

```
def isPalindrome(theString):
    if len(theString) == 0 or len(theString) == 1:
        # BASE CASE
        return True
    else:
        # RECURSIVE CASE
        ❶ head = theString[0]
        ❷ middle = theString[1:-1]
```